

ESTRUCTURA DE DATOS TP3

Nombre: Lucas Gaiton
PROFESOR : HECTOR REINAGA

Contenido

3. Generar en forma aleatoria 100 enteros comprendidos entre 1000 y 9999 y almacenarlos en un archivo. A partir de ese archivo, guardar los datos en una tabla hash, utilizando el método de resolución de colisiones rehashing (prueba lineal o secuencial), con las siguientes fórmulas para la transformación de claves:	2
Desarrollo:	2
b) Método de división o módulo.....	2
c) Método del medio cuadrado.....	4
4. A partir del archivo del ejercicio anterior, guardar los datos en una tabla hash, utilizando la función $H(K) = (7K + 1) \% 10$, con los siguientes métodos de resolución de colisiones:	5
Desarrollo:	5
b) Rehashing o reasignación con cuadrática.....	5
c) Rehashing o reasignación con doble dirección hash:.....	7

3. Generar en forma aleatoria 100 enteros comprendidos entre 1000 y 9999 y almacenarlos en un archivo. A partir de ese archivo, guardar los datos en una tabla hash, utilizando el método de resolución de colisiones rehashing (prueba lineal o secuencial), con las siguientes fórmulas para la transformación de claves:

- a) Función de restas sucesivas.
- b) Método de división o módulo.
- c) Método del medio cuadrado.
- d) Función truncamiento
- e) Método superposición por desplazamiento.
- f) Método superposición por plegamiento.

Desarrollo:

b) Método de división o módulo.

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.*;

public class TablaHash {

    private final int tableSize = 120;
    private int[] tabla;

    public TablaHash() {
        this.tabla = new int[this.tableSize];
        //Inicializamos la hash table con un entero especial que representa los espacios vacios
        for (int i = 0; i < this.tableSize; i++) {
            this.tabla[i] = -1;
        }
    }

    private int hash(int key) {
        if(key % this.tableSize >= this.tableSize)
```

```

        System.out.println("Lapu");
        return key % this.tableSize;
    }

    public void insert(int key) {
        int index = this.hash(key);
        while(this.tabla[index] != -1) {
            if (index + 1 > this.tableSize-1) {
                index = 0;
            } else {
                index++;
            }
        }
        this.tabla[index] = key;
    }

    public void mostrarHashTable() {
        System.out.println("Tabla Hash:");
        for (int i = 0; i < this.tableSize; i++) {
            if (this.tabla[i] != -1) {
                System.out.println("Índice " + i + ": " + this.tabla[i]);
            } else {
                System.out.println("Índice " + i + ": vacío");
            }
        }
    }

    public void cargarNumsArchivo(String fileName) throws IOException {
        File archivo = new File(fileName);
        Scanner scan = new Scanner(archivo);
        int cont = 0;
        while (scan.hasNextInt()) {
            System.out.println(cont + 1);
            int num = scan.nextInt();
            this.insert(num);
            cont++;
        }
        scan.close();
    }
}

```

```

public static void archivoNumerosAleatorios(String fileName) throws IOException {
    Random random = new Random();
    FileWriter writer = new FileWriter(fileName);

    for (int i = 0; i < 100; i++) {
        int num = 1000 + random.nextInt(9000);
        writer.write(num + "\n");
    }

    writer.close();
    System.out.println("Archivo creado");
}

public static void main(String[] args) {
    try {
        String nombre = "nombreDelArchivo";
        archivoNumerosAleatorios(nombre);
        TablaHash tabla = new TablaHash();
        tabla.cargarNumsArchivo(nombre);
        tabla.mostrarHashTable();
    } catch (Exception e) {
        System.out.println(e.getMessage());
    }
}

```

c) Método del medio cuadrado.

```

//Solo modificamos la función hash
private int hash(int key) {
    int cuadrado = (int) Math.pow(key, 2);
    String aux = String.valueOf(cuadrado);
    int particion = (int) aux.length() / 2;
    particion = particion - 1;
    String[] digitos = aux.split("");
    System.out.println("Esta es la key " + key + " Esta es el cuadrdo " + cuadrado + " y
esta es la particion " + particion);

    if (digitos[particion] == "0") {
        return Integer.parseInt(digitos[particion + 1]);
    } else {
        return Integer.parseInt(digitos[particion] + digitos[particion + 1]);
    }
}

```

4. A partir del archivo del ejercicio anterior, guardar los datos en una tabla hash, utilizando la función $H(K) = (7K + 1) \% 10$, con los siguientes métodos de resolución de colisiones:

a) Rehashing o reasignación con prueba lineal o secuencial.

b) Rehashing o reasignación con cuadrática.

c) Rehashing o reasignación con doble dirección hash.

d) Arreglos anidados o Cubos y secuencial o lineal.

e) Encadenamiento o tablas hash abiertas.

f) Zona de desbordamiento.

Luego, analizar cuál método es adecuado según el tamaño y los datos de la tabla hash, de acuerdo a la función pedida. Evaluar la cantidad de comparaciones e intercambios.

Desarrollo:

b) Rehashing o reasignación con cuadrática.

```
import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.*;

public class TablaHash {

    private final int tableSize = 10;
    private int[] tabla;

    public TablaHash() {
        this.tabla = new int[this.tableSize];
        //Inicializamos la hash table con un entero especial que representa los espacios vacios
        for (int i = 0; i < this.tableSize; i++) {
            this.tabla[i] = -1;
        }
    }

    //Solo modificamos el hash
    private int hash(int key) {
        return (7 * key + 1) % 10;
    }
}
```

```

    }

    public void insert(int key) {
        int index = this.hash(key);

        //Este es el framente de codigo que hay que modificar si se quiere modificar el
rehashing

        int cont = 1;
        while (this.tabla[index] != -1 ) {
            cont = (int)Math.pow(cont, 2);
            if (index + cont > this.tableSize - 1) {
                index = 0;
            } else {
                index = index+cont;
                cont++;
            }
        }
        this.tabla[index] = key;
    }

    public void mostrarHashTable() {
        System.out.println("Tabla Hash:");
        for (int i = 0; i < this.tableSize; i++) {
            if (this.tabla[i] != -1) {
                System.out.println("Índice " + i + ": " + this.tabla[i]);
            } else {
                System.out.println("Índice " + i + ": vacío");
            }
        }
    }

    public void cargarNumsArchivo(String fileName) throws IOException {
        File archivo = new File(fileName);
        Scanner scan = new Scanner(archivo);
        int cont = 0;
        while (scan.hasNextInt()) {
            System.out.println(cont + 1);
            int num = scan.nextInt();
            this.insert(num);
            cont++;
        }
    }

```

```

        scan.close();
    }

    public static void archivoNumerosAleatorios(String fileName) throws IOException {
        Random random = new Random();
        FileWriter writer = new FileWriter(fileName);
        for (int i = 0; i < 6; i++) {
            int num = 1000 + random.nextInt(9000);
            writer.write(num + "\n");
        }
        writer.close();
        System.out.println("Archivo creado");
    }

    public static void main(String[] args) {
        try {
            String nombre = "nombreDelArchivo";
            archivoNumerosAleatorios(nombre);
            TablaHash tabla = new TablaHash();
            tabla.cargarNumsArchivo(nombre);
            tabla.mostrarHashTable();
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}

```

c) Rehashing o reasignación con doble dirección hash:

```

import java.io.File;
import java.io.FileWriter;
import java.io.IOException;
import java.util.*;

```

```

public class TablaHash {

    private final int tableSize = 10;

    private int[] tabla;

    public TablaHash() {
        this.tabla = new int[this.tableSize];

        //Inicializamos la hash table con un entero especial que representa los espacios vacios
        for (int i = 0; i < this.tableSize; i++) {
            this.tabla[i] = -1;
        }
    }

    //Solo modificamos el hash
    private int hash(int key) {
        System.out.println("Esta es la key "+key+" esta es el resultado del hash "+(7 * key + 1)
% 10);
        return (7 * key + 1) % 10;
    }

    private int hash2(int key){
        return key%this.tableSize;
    }

    public void insert(int key) {
        int index = this.hash(key);

        //Este es el fragmento de codigo que hay que modificar si se quiere modificar el
rehashing
        // int cont = 1;

        do{
            if(this.tabla[index] != -1){
                index = this.hash2(key);
                while(this.tabla[index] != -1){
                    index++;
                }
            }
        }while(this.tabla[index] != -1 );

        this.tabla[index] = key;
    }
}

```

```

    }

    public void mostrarHashTable() {
        System.out.println("Tabla Hash:");
        for (int i = 0; i < this.tableSize; i++) {
            if (this.tabla[i] != -1) {
                System.out.println("Índice " + i + ": " + this.tabla[i]);
            } else {
                System.out.println("Índice " + i + ": vacío");
            }
        }
    }
}

public void cargarNumsArchivo(String fileName) throws IOException {
    File archivo = new File(fileName);
    Scanner scan = new Scanner(archivo);

    while (scan.hasNextInt()) {
        int num = scan.nextInt();
        this.insert(num);
    }
    scan.close();
}

public static void archivoNumerosAleatorios(String fileName) throws IOException {
    Random random = new Random();
    FileWriter writer = new FileWriter(fileName);
    for (int i = 0; i < 6; i++) {
        int num = 1000 + random.nextInt(9000);
        writer.write(num + "\n");
    }
    writer.close();
    System.out.println("Archivo creado");
}

public static void main(String[] args) {
    try {
        String nombre = "nombreDelArchivo";
        archivoNumerosAleatorios(nombre);
        TablaHash tabla = new TablaHash();
    }
}

```

```
        tabla.cargarNumsArchivo(nombre);  
        tabla.mostrarHashTable();  
    } catch (Exception e) {  
        System.out.println(e.getMessage());  
    }  
}  
  
}
```